

基于 TCP Socket 与 select 的多人聊天室系统

《Linux 高级编程》课程设计报告

作者：曹博雄 学号：2023102959 日期：2026 年 7 月

项目仓库：<https://github.com/CBx-2023/linux-select-chatroom>

GitHub Pages 文档：<https://cbx-2023.github.io/linux-select-chatroom/>

摘要

本文设计并实现了一个运行在 Linux 系统上的多人聊天室系统。系统使用 C++ 编写，基于 TCP Socket 完成客户端与服务端之间的可靠通信，服务端采用单进程 select I/O 多路复用模型同时管理多个客户端连接。客户端连接服务端后使用昵称登录，支持群聊广播、用户私聊、在线用户列表查询、正常退出、异常断开处理和服务端日志记录。项目重点体现 Linux 网络编程中的文件描述符管理、Socket 生命周期、文本协议解析、TCP 字节流拆包、异常清理和可验证测试。实验结果表明，该系统能够在 Linux 环境下完成多客户端并发聊天，并能在非法命令、重复昵称、超长消息和连接异常等情况下保持服务端可用。

关键词：Linux；TCP Socket；select；I/O 多路复用；多人聊天室；C++

1 引言

《Linux 高级编程》课程设计要求在 Linux 系统上实现一个具有服务端和客户端的软件系统，并体现 Socket 编程或进程通信等高级编程内容。多人聊天室是典型的网络类应用：服务端需要监听客户端连接，接收某个客户端发送的消息，并将消息分发给其他客户端；客户端需要连接服务端，发送用户输入，并实时接收服务端转发的消息。

本文实现的 TCP Select Chatroom 选择 TCP 作为传输协议，使用 select 作为 I/O 多路复用机制。相比为每个客户端创建线程，单进程 select 模型更适合展示 Linux 文件描述符和事件循环的核心思想，也便于在课程设计规模内控制代码复杂度。

本项目的目标包括：

- 实现一个 TCP 聊天室服务端和一个 TCP 聊天室客户端；
- 服务端使用单进程加 select 管理多个客户端连接；
- 支持昵称登录、群聊广播、私聊、在线用户列表和退出；
- 处理重复昵称、非法命令、超长消息和异常断开；
- 将启动、登录、退出、异常断开、群聊、私聊和错误事件写入日志。

为保持范围清晰，本系统不实现账号密码认证、数据库存储、离线消息、历史记录查询、图形界面和管理员踢人功能。

2 系统设计

2.1 总体架构

系统由两个可执行程序组成：server 和 client。服务端负责监听端口、接受连接、维护在线用户、解

析命令、分发消息和写入日志；客户端负责连接服务端、发送登录命令、读取用户输入、接收服务端消息并在终端显示。

图 1 给出系统结构。协议层提供命令解析、响应生成和按行拆包；服务端状态层维护 fd、昵称和登录状态；服务端业务层负责连接管理与消息路由；客户端交互层将用户输入转换成协议命令，并将服务端响应格式化为可读文本。

2.2 代码组织

项目代码位于 code/ 目录，主要结构如表 1 所示。

表 1：主要文件与职责

文件	作用
protocol.*	协议解析、响应生成、按行缓冲
server.*	服务端监听、select 循环、消息路由
client_manager.*	在线用户状态管理
logger.*	服务端运行日志记录
client.*	客户端连接、登录和交互循环
input_handler.*	用户输入到协议命令的转换
display.*	服务端响应的终端显示格式化

3 TCP Socket 通信

TCP 是面向连接、可靠、有序的字节流协议。聊天室要求客户端和服务端之间保持长连接，并保证消息按顺序到达，因此使用 TCP 作为传输层协议。

服务端启动时依次调用 socket、setsockopt、bind 和 listen。当监听 fd 可读时，调用 accept 接收新客户端。核心代码如代码 1 所示。

客户端终端	ChatClient	TCP 文本协议	ChatServer
用户输入	InputHandler	LOGIN/MSG/PRIVATE LIST/QUIT	ClientManager
消息显示	Display	OK/ERROR CHAT/SYSTEM	Logger

图 1: 系统总体架构

```

1 listen_fd_ = socket(AF_INET, SOCK_STREAM, 0);
2 setsockopt(listen_fd_, SOL_SOCKET, SO_REUSEADDR,
3           &reuse, sizeof(reuse));
4 bind(listen_fd_, reinterpret_cast<sockaddr*>(&address),
5       sizeof(address));
6 listen(listen_fd_, kListenBacklog);

```

Listing 1: 服务端 Socket 初始化

客户端启动后解析服务端 IP、端口和昵称，创建 TCP Socket，再调用 `connect` 建立连接。连接成功后立即发送 LOGIN <nickname>，服务端返回 OK 后客户端进入交互循环。

由于 TCP 是字节流协议，一次 `recv` 可能读到半条消息，也可能读到多条消息。因此系统约定每条协议消息以换行符结束，并为每个连接维护 `LineBuffer`。该缓冲区将新收到的字节追加到内部字符串，遇到换行符时输出完整消息，未形成完整行的部分继续保留。

4 select I/O 多路复用

4.1 服务端事件循环

服务端采用单进程 `select` 模型。每次进入事件循环时，服务端重新构造 `fd_set`，将监听 fd 和所有客户端 fd 加入集合，然后调用 `select` 等待可读事件。若监听 fd 可读，说明有新连接；若客户端 fd 可读，说明该客户端发送了消息或已经断开。

```

1 FD_ZERO(&read_fds);
2 FD_SET(listen_fd_, &read_fds);
3 int max_fd = listen_fd_;
4
5 for (int fd : clients_fds()) {
6     FD_SET(fd, &read_fds);
7     if (fd > max_fd) {
8         max_fd = fd;
9     }
10 }
11
12 int ready = select(max_fd + 1, &read_fds,
13                   nullptr, nullptr, timeout_ptr);

```

Listing 2: 服务端 select 主循环片段

该模型的优势在于：服务端不需要为每个客户端创建线程，所有连接通过 fd 集合统一调度，资源管理逻辑集中，适合课程设计中展示 Linux 文件描述符和 I/O 多路复用机制。

4.2 客户端事件循环

客户端也使用 `select` 同时监听标准输入和服务端 Socket。标准输入可读时，客户端读取用户命令并发送协议消息；服务端 Socket 可读时，客户端读取服务端响应并格式化输出。这样可以在不使用线程的情况下实现“边输入边接收”。

5 通信协议

系统使用按行分隔的文本协议。文本协议便于调试，也便于使用终端或脚本模拟客户端。客户端命令和服务端响应如表 2 所示。

表 2: 聊天室文本协议

方向	消息格式
客户端	LOGIN <nickname>
客户端	MSG <content>
客户端	PRIVATE <target> <content>
客户端	LIST
客户端	QUIT
服务端	OK <message>
服务端	ERROR <reason>
服务端	SYSTEM <message>
服务端	CHAT <nickname> <content>
服务端	PRIVATE <from> <content>
服务端	USERLIST <name1>,<name2>,...

协议约束包括：客户端连接后必须先发送 LOGIN；昵称长度为 1 到 20 个字符，只允许英文字母、数字和下划线；单条协议消息最大 1024 字节；未登录客户端只能发送 LOGIN 或 QUIT。

```

1 bool is_valid_nickname(const std::string& nickname) {
2     if (nickname.empty() ||
3         nickname.size() > kMaxNicknameLength) {
4         return false;
5     }
6
7     for (char c : nickname) {
8         if (c != '_' && !is_ascii_letter_or_digit(c)) {
9             return false;
10        }
11    }
12    return true;
13 }

```

Listing 3: 昵称合法性检查

6 服务端实现

6.1 客户端状态管理

服务端使用 `ClientManager` 管理客户端状态。每个客户端记录 fd、昵称、登录状态和接收缓冲区。状态管理同时支持按 fd 查找和按昵称查找：按 fd 查找用于处理 Socket 事件，按昵称查找用于私聊和重复昵称检测。

客户端连接后先处于未登录状态。服务端只允许未登录客户端发送 LOGIN 或 QUIT。收到 LOGIN 后，服务端检查昵称是否合法且未被占用，再写入昵称和

登录状态。登录成功后，服务端向该客户端返回 OK login successful,并向其他在线用户广播上线通知。

6.2 消息路由

群聊消息由 handle_group_message 处理。服务端确认发送者已登录后，生成 CHAT <nickname> <content> 响应，并遍历所有已登录客户端，将消息发送给除发送者以外的用户。

私聊消息由 handle_private_message 处理。服务端根据目标昵称查找目标客户端；目标不存在时返回 ERROR user not online；目标存在时仅向目标 fd 发送 PRIVATE <from> <content>，不会泄露给其他客户端。

```
1 const ClientInfo* target_client =
2   clients_.find_by_nickname(target);
3 if (target_client == nullptr ||
4     !target_client->logged_in) {
5     send_response(fd, make_error("user not online"));
6     return;
7 }
8
9 send_response(target_client->fd,
10              make_private(sender->nickname, content));
```

Listing 4: 私聊消息路由片段

6.3 异常处理与日志

服务端需要处理重复昵称、非法昵称、未登录命令、非法命令、私聊目标不存在、超长消息、客户端异常断开、send 或 recv 失败等情况。对每个异常，服务端尽量只清理受影响连接，不影响其他在线用户。

Logger 将关键事件追加写入 server.log，事件类型包括 STARTUP、LOGIN、QUIT、ABNORMAL_DISCONNECT、GROUP_CHAT、PRIVATE_CHAT 和 ERROR。日志写入失败时，服务端只向标准错误输出提示，不因为日志问题终止。

7 客户端实现

7.1 启动与登录

客户端启动格式如下：

Listing 5: 客户端启动命令

```
./client 127.0.0.1 8888 alice
```

客户端解析服务端 IP、端口和昵称，连接成功后发送 LOGIN alice。如果服务端返回 OK，客户端进入交互循环；如果返回 ERROR 或关闭连接，客户端输出失败原因并退出。

7.2 用户输入处理

InputHandler 将用户输入转换为协议命令，如表 3 所示。空输入会被忽略；/w 缺少目标或消息内容时，客户端在本地提示错误，不向服务端发送非法命令。

表 3: 客户端输入映射

用户输入	协议命令
hello	MSG hello
/w bob hello	PRIVATE bob hello
/list	LIST
/quit	QUIT

7.3 消息显示

服务端响应是协议文本，直接显示可读性较差。客户端使用 Display 将响应转换为终端友好的格式。例如，CHAT alice hello 显示为 [group] alice: hello,PRIVATE bob hi 显示为 [private] bob: hi, ERROR user not online 显示为 Error: user not online。

8 实验与验证

8.1 构建方法

项目提供 Makefile。在 code/ 目录执行 make 后生成 server 和 client 两个可执行文件。执行 make clean 可清理构建产物。

Listing 6: 构建与运行命令

```
cd code
make
./server 8888
./client 127.0.0.1 8888 alice
```

8.2 测试覆盖

本项目包含自动化测试和端到端手动测试。自动化测试覆盖协议解析、客户端状态管理、日志写入、客户端输入转换、显示格式化、客户端连接、客户端 select 循环、服务端监听、登录生命周期、消息路由和错误处理。主要测试结果如表 4 所示。

表 4: 测试覆盖摘要

测试文件	覆盖内容
protocol_tests	拆包、昵称、命令、1024 字节限制
server_routing_tests	群聊、私聊、在线列表、日志
server_lifecycle_tests	登录、退出、异常断开
server_hardening_tests	非法命令、超长消息、send/recv 失败
client_select_loop_tests	标准输入和服务端 Socket 并发处理

重新整理本文时，已执行 make -C code clean && make -C code，服务端和客户端均能成功编译。随后编译并运行 11 个 C++ 测试二进制，测试进程均以退出码 0 结束。端到端验证还检查了三客户端登录、群聊广播、私聊只到目标、在线列表、重复昵称拒绝、非法命令、超长消息、正常退出、异常断开和日志记录。

9 结论

本文实现了一个基于 Linux TCP Socket 和 `select` 的多人聊天室系统。系统通过按行文本协议实现客户端和服务端协作, 通过 `ClientManager` 维护在线用户状态, 通过服务端消息路由逻辑完成群聊、私聊和在线列表, 通过 `Logger` 记录关键运行事件, 并通过客户端 `select` 循环实现用户输入和服务端消息的并发处理。

课程设计表明, `select` 能够以较低复杂度支持多个客户端连接, 并能清晰展示 Linux 文件描述符、Socket 生命周期和 I/O 多路复用机制。后续可以扩展账号认证、持久化聊天记录、图形界面、管理员功能, 或者将服务端事件模型替换为更适合大量连接的 `epoll`。

参考文献

- [1] W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff. *UNIX Network Programming, Volume 1: The Sockets Networking API*. Addison-Wesley, 3rd edition, 2003.
- [2] Michael Kerrisk. *The Linux Programming Interface*. No Starch Press, 2010.
- [3] IEEE and The Open Group. *The Open Group Base Specifications Issue 7, POSIX.1-2017*. 2018 edition.
- [4] Linux man-pages project. Manual pages for `socket(2)`, `select(2)`, `recv(2)` and `send(2)`.